

Dokumentacja wstępna — Zadanie 21

WAE 2026L: Losowa inicjalizacja ścieżki ewolucyjnej p_σ w CMA-ES

Sebastian Rydz, Jerzy Czarnecki

Prowadzący: mgr inż. Eryk Warchulski

21 kwietnia 2026

1 Opis problemu i sposób rozwiązania

Algorytm CMA-ES utrzymuje tzw. ścieżkę ewolucyjną $p_\sigma \in \mathbb{R}^n$ — wektor akumulujący znormalizowane kroki średniej populacji między pokoleniami. Służy on do adaptacyjnego strojenia zasięgu mutacji σ poprzez porównanie oczekiwanej normy wektora losowego $\mathcal{N}(\mathbf{0}, \mathbf{I})$ z aktualną normą $\|p_\sigma\|$. Standardowo $p_\sigma^{(0)} = \mathbf{0}$, co w pierwszych iteracjach powoduje sztuczne zaniżenie $\|p_\sigma\|$ i w konsekwencji zbyt agresywne zmniejszanie σ , zanim ścieżka zdąży się „rozpędzić”.

Celem projektu jest zbadanie, jak zachowuje się algorytm, gdy $p_\sigma^{(0)}$ zainicjalizujemy *ważoną sumą losowych zaburzeń* zamiast wektorem zerowym. Zakładamy interpretację najprostszą i spójną z mechaniką CMA-ES: wagi w_i to te same wagi rekombinacyjne, których algorytm używa do aktualizacji średniej, a $z_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$p_\sigma^{(0)} = \sqrt{\mu_{\text{eff}}} \sum_{i=1}^{\mu} w_i z_i, \quad \sum_{i=1}^{\mu} w_i = 1, \quad w_i > 0.$$

Czynnik $\sqrt{\mu_{\text{eff}}}$ sprawia, że $p_\sigma^{(0)}$ ma rozkład $\mathcal{N}(\mathbf{0}, \mathbf{I})$ — taki sam, jaki algorytm zakłada dla p_σ w stanie stacjonarnym pod losowym selektorem. Proponowana modyfikacja jest więc probabilistycznie „neutralna” w sensie rozkładu, ale zmienia dynamikę pierwszych iteracji. Dodatkowo, zgodnie z treścią zadania, rozważymy różne wartości początkowe zasięgu mutacji $\sigma^{(0)}$.

Badanie sprowadza się do porównania:

- **CMA-ES (baseline):** $p_\sigma^{(0)} = \mathbf{0}$, różne $\sigma^{(0)}$,
- **CMA-ES-RI** (*Randomized Initialization*): $p_\sigma^{(0)}$ wg wzoru powyżej, te same $\sigma^{(0)}$.

Hipotezy badawcze. (H1) Modyfikacja najwyraźniej wpłynie na bardzo wczesne iteracje ($t < 1/c_\sigma$), ponieważ współczynnik dyskontowania $(1-c_\sigma)$ stosunkowo szybko „zapomina” stan początkowy. (H2) Efekt zależy od $\sigma^{(0)}$: dla zbyt małego $\sigma^{(0)}$ losowa inicjalizacja może pomóc (zapobiega przedwczesnemu szrinkingowi), dla dużego $\sigma^{(0)}$ może nie mieć znaczenia lub wręcz zaszkodzić. (H3) Na funkcjach wielomodalnych (Rastrigin, Ackley, Griewank) wariancja wyników będzie wyższa niż na unimodalnych (Sphere, Rosenbrock), co wymaga większej liczby powtórzeń do oceny istotności.

2 Planowane eksperymenty numeryczne

Funkcje testowe. Klasyczny zestaw: Sphere (unimodalna, wypukła), Rosenbrock (unimodalna, dolinowa), Rastrigin, Ackley, Griewank (wielomodalne). Wymiarowości $n \in \{2, 10, 30\}$ — pozwala to obserwować skalowanie efektu modyfikacji z wymiarem, co jest istotne, bo c_σ w CMA-ES jawnie zależy od n .

Hiperparametry. Populacja $\lambda = 4 + \lfloor 3 \ln n \rfloor$ (domyślna CMA-ES), $\mu = \lfloor \lambda/2 \rfloor$, wagi rekombinacyjne domyślne z `pycma`. Dla $\sigma^{(0)}$ przeskanujemy siatkę $\{0.1, 0.5, 1.0, 2.0\} \cdot L$, gdzie L to połowa szerokości dziedziny. Punkt startowy $m^{(0)}$ losowany jednostajnie z dziedziny (ten sam seed dla obu wariantów w danym powtórzeniu, aby wyizolować efekt inicjalizacji p_σ).

Protokół. 31 niezależnych powtórzeń na każdą kombinację (funkcja, n , $\sigma^{(0)}$, wariant). Budżet: $10^4 \cdot n$ ewaluacji funkcji celu na uruchomienie lub osiągnięcie precyzji 10^{-8} . Rejestrujemy: krzywą zbieżności (najlepsze f vs. liczba ewaluacji), trajektorię $\sigma^{(t)}$ oraz $\|p_\sigma^{(t)}\|$, liczbę ewaluacji do osiągnięcia kilku progów precyzji (10^{-2} , 10^{-5} , 10^{-8}), oraz współczynnik powodzenia (success rate).

Analiza. Porównanie statystyczne parami (wariant baseline vs. RI) przy ustalonym seedzie: test Wilcoxona dla par związanych na liczbie ewaluacji do progu; Empirical Runtime Distributions (ERT) do wizualizacji. Szczególną uwagę poświęcimy *fazie przejściowej* pierwszych iteracji — to tam oczekujemy różnic, a obecne w literaturze analizy CMA-ES często ją pomijają.

Reprodukowalność. Każde powtórzenie parametryzowane deterministycznym seedem PRNG (`numpy.random.Generator`, PCG64). Lista seedów składowana w repozytorium. Skrypt `reproduce.sh` odtwarza wszystkie uruchomienia i wykresy; ciężkie surowe dane (CSV/parquet) w razie przekroczenia 5 MB trafiają poza repo (OneDrive), w repo pozostanie manifest z sumami kontrolnymi.

3 Wybór technologii

Python 3.12 jako główny język — wygodna analiza statystyczna i standard w społeczności ewolucyjnej. **pyma** (referencyjna implementacja Hansena) jako bazowy CMA-ES; modyfikacja poprzez rozszerzenie, nie fork — podmieniamy tylko inicjalizację p_σ i udostępniamy ją jako opcję konstruktora. Pozwala to oddzielić badany efekt od nieintencjonalnych różnic implementacyjnych. **NumPy** / **SciPy** do arytmetyki i testów statystycznych, **Matplotlib** + **seaborn** do wykresów, **pandas** / **pyarrow** do agregacji wyników, **tqdm** i **joblib** do zrównoleglenia powtórzeń. Zarządzanie zależnościami: `uv + pyproject.toml`. Konteneryzacja: **Podman** z obrazem opartym o `python:3.12-slim`. Repozytorium: **Git**, z wymuszonym stylem **Ruff** + **Black**. Środowisko docelowe: Linux (x86_64), zweryfikowane pod Ubuntu 24.04.